

POLITECNICO
MILANO 1863

SPIN

Formal Verification of Concurrent Systems

Matteo Gatti
Alessio Ginolfi
Samuele Grisoni

Formal methods for software engineering (UIC 540)
Academic Year 2025-2026

Foundations

- 1) [Model checking](#)
- 2) [History](#)
- 3) [Real world applications](#)

SPIN Basics

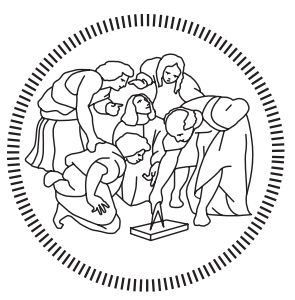
- 4) [Introduction to SPIN](#)
- 5) [Büchi automaton in SPIN](#)

How it works

- 6) [Search algorithms](#)
- 7) [PROMELA language](#)
- 8) [Tool setup](#)

Further topics

- 9) [Extensions](#)
- 10) [Demo preview](#)
- 11) [Bibliography](#)



POLITECNICO
MILANO 1863

Model checking

Concurrent systems

In concurrent systems, multiple processes execute in **parallel** and interact with each other. These interactions are **non-deterministic**, so the exact execution order is unpredictable.

In this context, bugs can be hard to detect, and testing is usually not enough since it explores only a subset of all possible executions.

Race conditions and **deadlocks** are two typical bugs in concurrent systems. In this example, we can see a race condition, because two processes access and modify the same variable without any synchronization. Since the increment of x is not atomic, the result depends on the order in which the operations are executed.

```
int x = 0;

process P1 {
    x = x + 1;
}

process P2 {
    x = x + 1;
}

// x = 1 or x = 2???
```

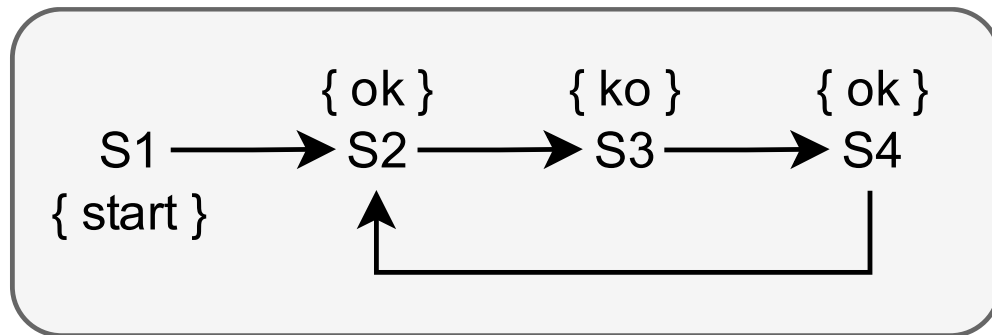


Model checking

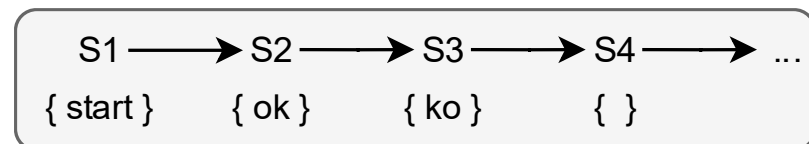
Model checking is an automated technique used to verify whether a **system** satisfies a given **property**.

The system is modeled as a **transition system**, which is similar to a finite-state automaton (FSA).

States represent system configurations and transitions represent possible executions. Each state is labeled with a set of **atomic propositions**, representing the properties that hold in that state.



We model **linear-time (LT)** properties using **Linear Temporal Logic (LTL)**. An LT property is a set of infinite traces (sequences of state labels) that describe a particular behaviour of the system.



Trace: {start, ok, ko, ...}

LT property: $\Box \neg \text{ko}$ (ko never occurs)

In this case, the property is violated

Model checking (2)

Advantages

- It explores all possible states of the system
- It is fully automated
- It provides counterexamples when a property is violated

Disadvantages

- State space explosion (high computational complexity)
- It requires a finite-state model



Traces and properties

In transition systems, we can have non-terminating (infinite) traces.

➡ **Traces(TS)** = set of all infinite traces of the transition system.

To define properties over TS we need also the concept of infinite word.

➡ A^ω = set of infinite words over the alphabet A.

Example of infinite word: {start, success}{success}{error} $\emptyset$ $\emptyset$ {success} ...

If **AP** is the set of atomic propositions of a transition system (i.e., the variables that can be true or false in each state), then a linear-time (LT) property is defined as a subset of $(2^{AP})^\omega$, called **P** in the box on the right.

A transition system TS satisfies a property P, written **TS** $\models$ **P**, if and only if **Traces(TS)** $\subseteq$ **P**.

Types of properties

Safety properties: LT properties such that any violating trace contains a **bad prefix**. Once a bad prefix occurs, the property is permanently violated.

Liveness properties: LT properties for which there are no bad prefixes. Unlike safety properties, it is not possible to detect a violation in a finite prefix, because the desired event might still happen later.

Examples

At most one process can be in the critical section at any time (**mutual exclusion**).

No process will wait forever to enter the critical section (**starvation freedom**).



Fairness assumptions

They are assumptions that restrict the set of possible executions of the system. They are often needed to prove liveness properties. We can divide them into three categories: **weak**, **strong** and **unconditional** fairness.

Examples

➡ Weak fairness

If a process **remains continuously** enabled from some point on, it cannot be postponed forever.

➡ Strong fairness

If a process is enabled **infinitely often**, it cannot be postponed forever.

➡ Unconditional fairness

Every process is scheduled infinitely often.

Unconditional fairness $\Rightarrow$ **Strong fairness** $\Rightarrow$ **Weak fairness** (from stronger to weaker assumptions).



Büchi Automaton

A Büchi automaton is a mathematical model similar to a transition system. The key difference is that some states are marked as **accepting states**, usually represented with double circles. An execution is accepted if it visits accepting states **infinitely often**.

Büchi automata are used to describe **infinite executions** of a system.

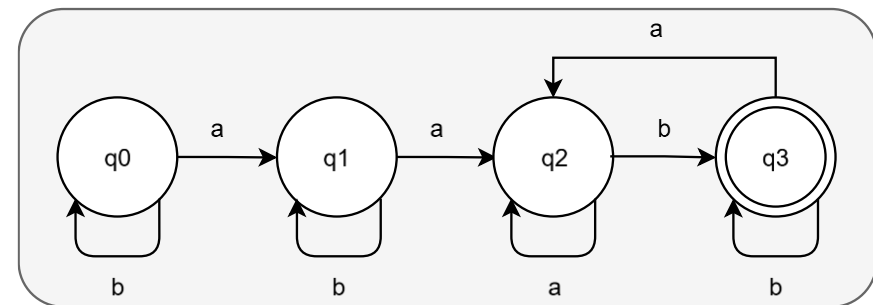
Example

Let $A = \{a, b\}$ be the alphabet.

This automaton accepts infinite words over A that contain at least two a 's and infinitely many b 's.

The language that we obtain is a **ω -language** (a language of infinite words).

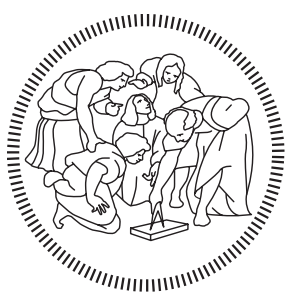
In verification, such a language is called an **ω -property**.



$a a b a b b a b b b \dots$ is accepted
 $a a b a a a a a \dots$ is not accepted

From theory to SPIN

Concurrent system	➡	modeled in PROMELA (Process Meta Language) (processes, variables, and communication between processes)
Transition system	➡	derived from the PROMELA model and explored implicitly on-the-fly (states and transitions are generated during the verification phase, not built in advance)
States & atomic propositions	➡	program variables and conditions (each state corresponds to a configuration of variables and process states)
Traces	➡	possible executions of the model (sequences of states generated by different interleavings)
LTL properties	➡	specified using LTL formulas (SPIN checks safety and liveness properties, but it does not assume fairness by default)
Büchi automaton	➡	LTL properties are translated into Büchi automaton
Counterexample	➡	error trace produced by SPIN



POLITECNICO
MILANO 1863

History

Introduction to SPIN Model Checker

SPIN is a powerful logic **model checking** tool designed to automatically detect software defects in **concurrent systems** by exhaustively analyzing all possible behaviours.

It was created by **Gerard J. Holzmann** (principal designer) at the Computing Sciences Research Center of **Bell Laboratories**. The SPIN system was developed in the 1980s and 1990s, while its specification language, PROMELA, first appeared in 1979.

In **concurrent systems**, process interleavings are uncontrollable and unobservable, which makes traditional software testing unable to reliably capture all possible execution scenarios. For this reason, **SPIN** was created to address this limitation, helping designers detect **subtle bugs**, **mismatched assumptions**, and **deadlocks** that regular testing would miss.

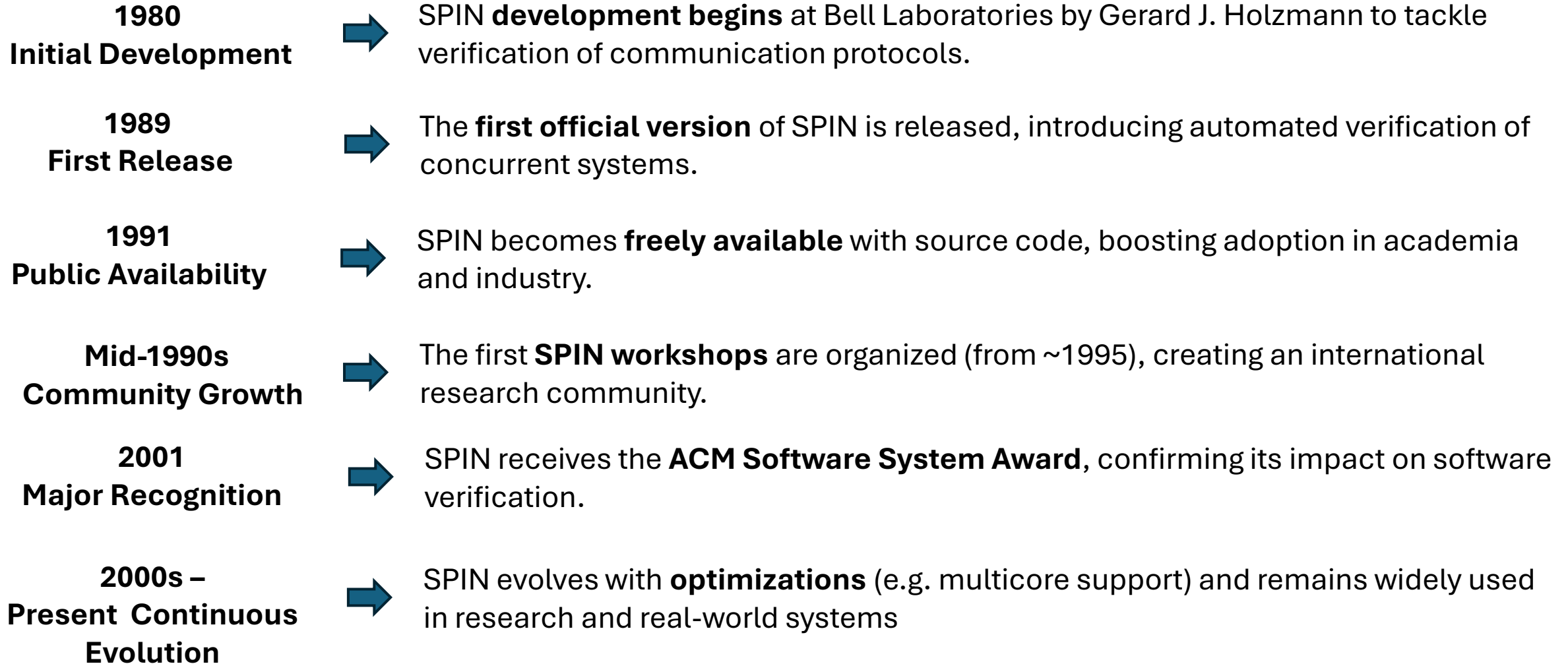


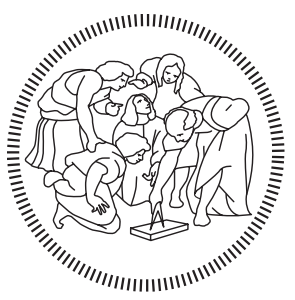
Spin Model Checker logo (1984-1995)



Bell Laboratories logo (1984-1995)

SPIN Model Checker Timeline





POLITECNICO
MILANO 1863

Real world applications

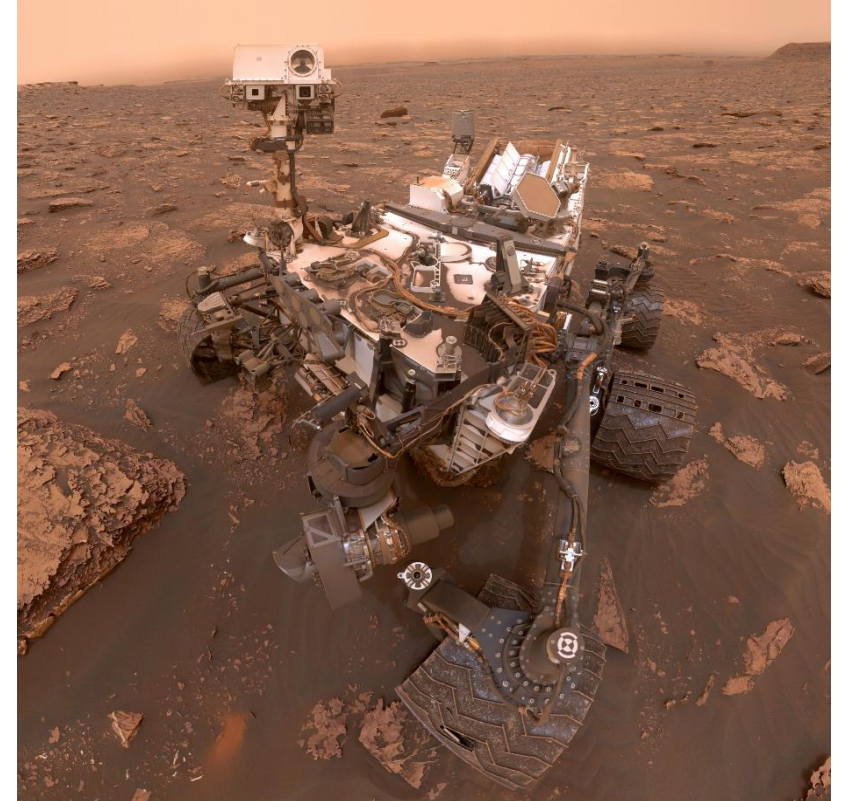
Mars Science Laboratory (NASA)

Description

The Mars Science Laboratory (MSL) is a mission by NASA launched in 2011 to explore Mars and study its past habitability. The main work is carried out by the Curiosity rover, which moves across the Martian surface collecting data and analyzing the environment.

The Role of SPIN

For the MSL mission, model checking was heavily used, primarily through SPIN. This tool was specifically applied to find and prevent race conditions caused by multithreading in the flight software. Ultimately, this process caught several concurrency flaws that had slipped through despite numerous code reviews.



NASA's Mars Curiosity June 13, 2018. (Image credit: NASA/JPL-Caltech)

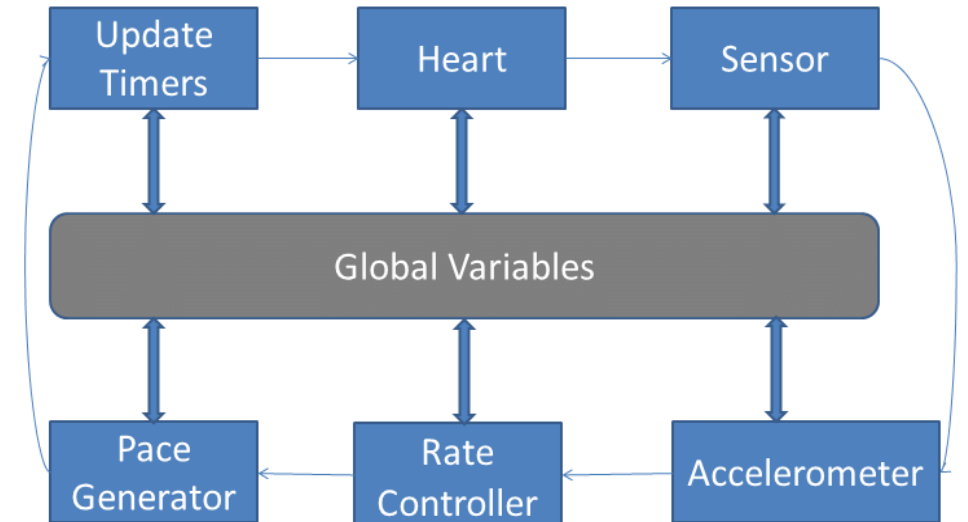
Pacemaker Challenge (Boston Scientific)

Description

Pacemakers are critical life-saving devices where any software failure can be deadly, making traditional testing methods insufficient to guarantee absolute safety. To address this, in 2007, a Boston Scientific pacemaker's detailed system specifications were made available to researchers. The goal was to use formal verification methods to translate these medical requirements into mathematical models, guaranteeing the complete absence of software defects before physical deployment.

The Role of SPIN

A notable approach to this challenge is presented by Asankhaya Sharma in “*End to End Verification and Validation with SPIN*”. The study uses the SPIN model checker to formally model both a cardiac pacemaker and an abstract heart environment.



Detailed model of pacemaker with various components

Lucent Pathstar Access Server (Lucent Technologies)

Description

The Lucent PathStar Access Server was an early device that combined traditional telephone services with internet routing in a single machine. Developed by Lucent Technologies, it helped pave the way for modern internet-based phone calls. The system was actively used from the late 1990s until it was abandoned in 2001.

The Role of SPIN

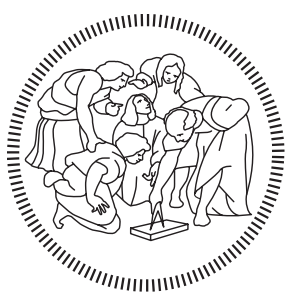
In the Lucent PathStar project, SPIN was used specifically to formally verify the concurrent C source code of the call-processing software to prevent critical deadlocks and race conditions. Gerard J. Holzmann, SPIN's creator, directly led this Bell Labs initiative by developing custom tools to automatically extract verification models from the code.

Lucent Technologies
Bell Labs Innovations



Lucent Technologies logo





POLITECNICO
MILANO 1863

Introduction to SPIN

SPIN - Simple Promela INterpreter

SPIN is a verification tool for concurrent and distributed systems, where system behavior is described using the PROMELA language.

Some of the main features are:

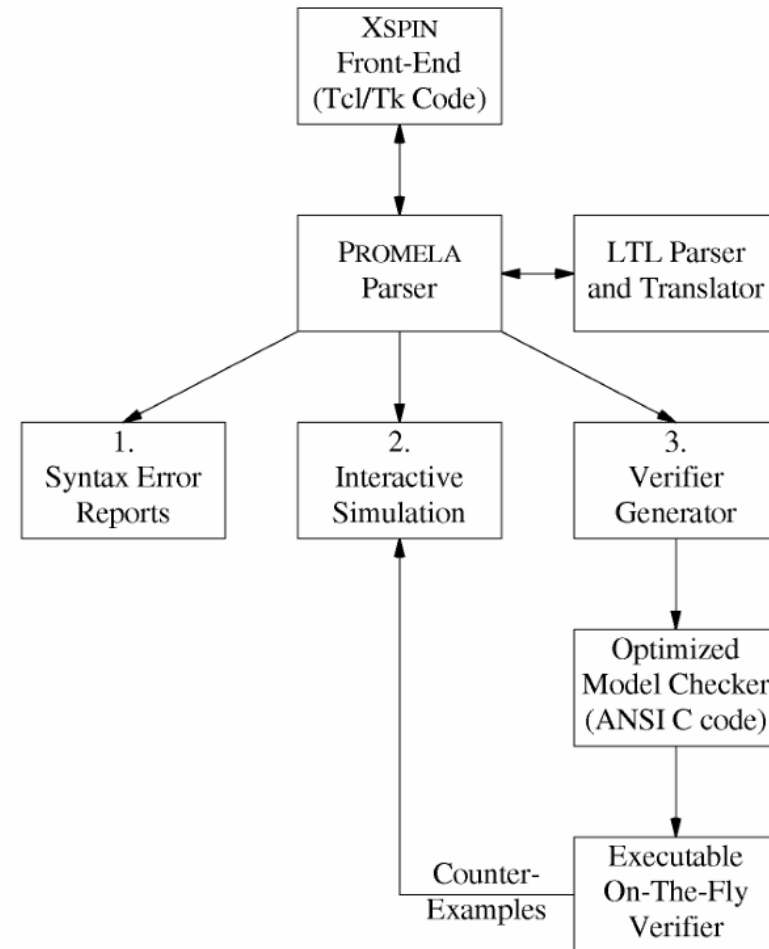
- **Automata-based** ➡ SPIN is an automata-based model checker. It represents both the system behavior and the properties to verify as automata.
- **On-the-fly Verification** ➡ SPIN explores the system step by step, generating only the necessary states of the system automaton during the analysis. This avoids building the entire automaton in advance, reducing memory and computation costs drastically.
- **Property Checking** ➡ Supports the verification of basic safety and liveness properties (with or without LTL), **LTL model checking**, assertions.

SPIN Model Checker Structure

The basic structure and typical workflow of the SPIN model checker can be represented by the architecture diagram on the right.

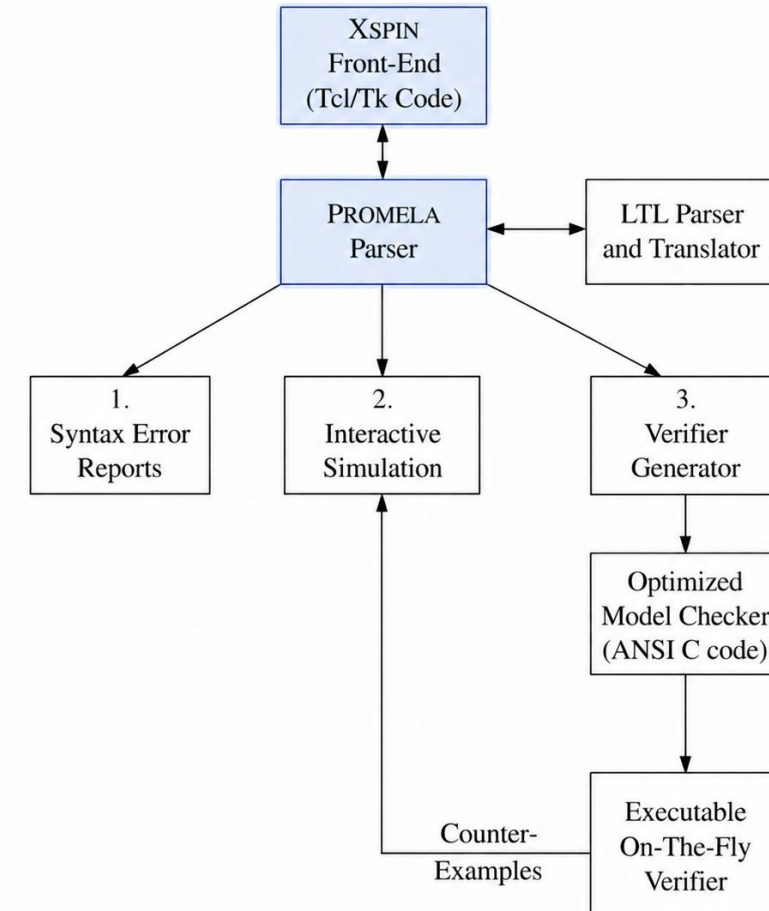
It shows an iterative feedback loop that guides a concurrent system from a high-level specification all the way to automated formal verification.

To make this process easier to understand in practice, the overall workflow can be broken down into five main phases.



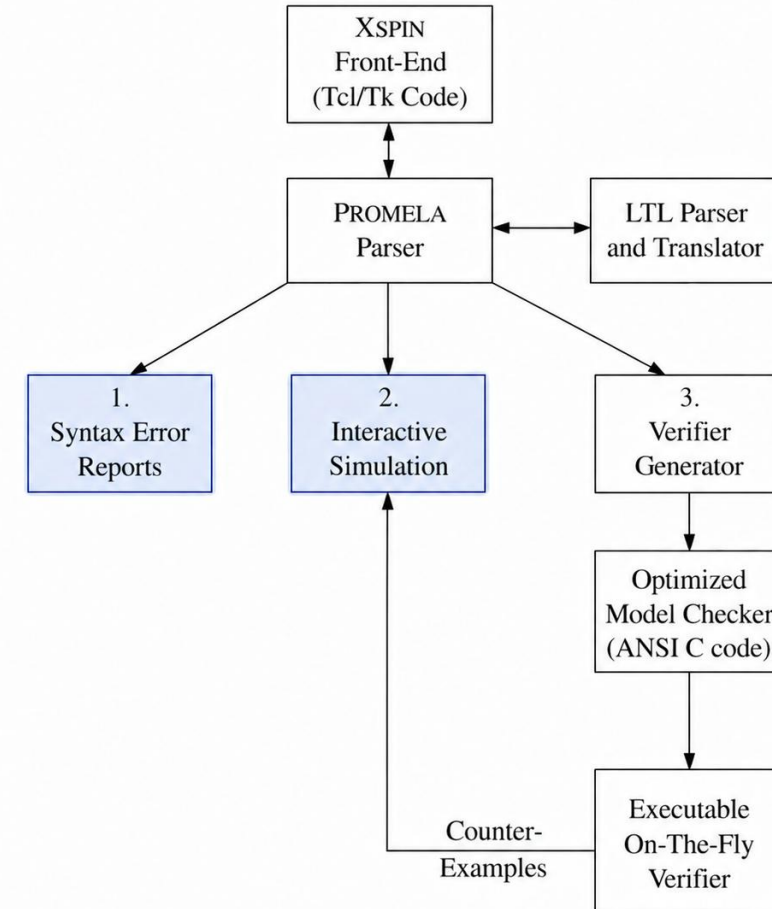
SPIN Model Checker Structure

1) Specification and Modelling



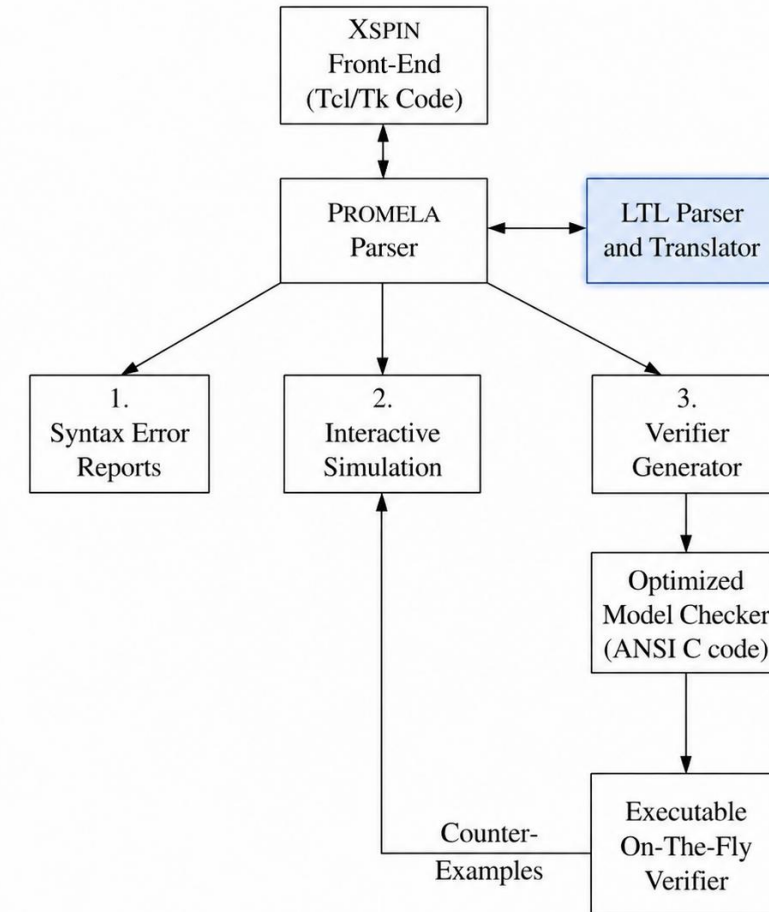
SPIN Model Checker Structure

- 1) Specification and Modelling
- 2) Syntax Check and Simulation**



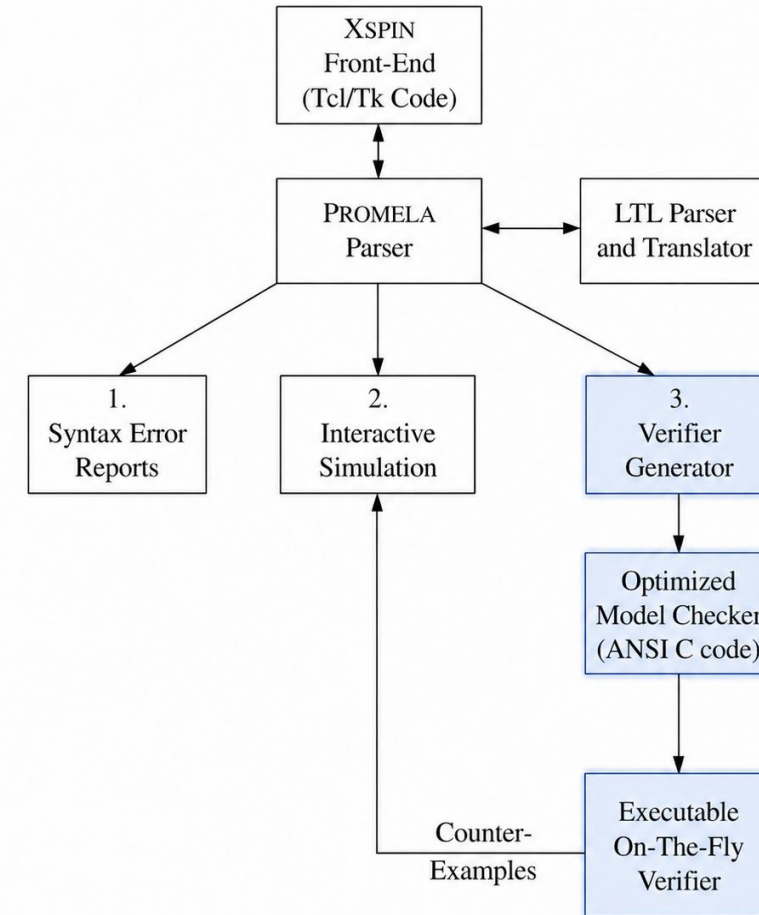
SPIN Model Checker Structure

- 1) Specification and Modelling
- 2) Syntax Check and Simulation
- 3) Requirements Formalization**



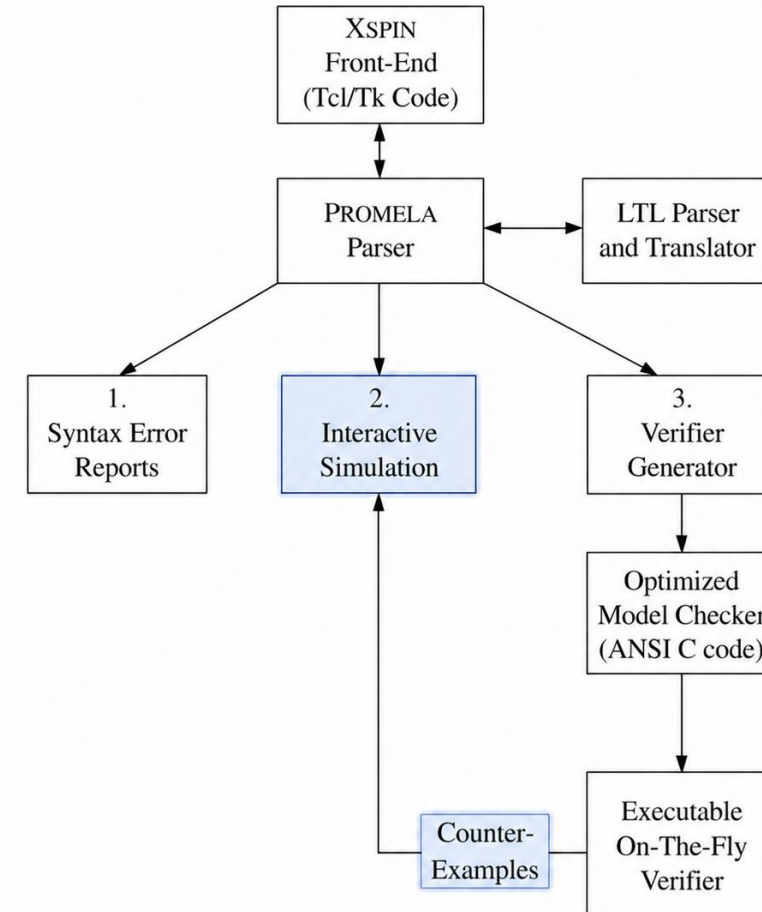
SPIN Model Checker Structure

- 1) Specification and Modelling
- 2) Syntax Check and Simulation
- 3) Requirements Formalization
- 4) Verifier Generation and Execution**



SPIN Model Checker Structure

- 1) Specification and Modelling
- 2) Syntax Check and Simulation
- 3) Requirements Formalization
- 4) Verifier Generation and Execution
- 5) **Counterexample Analysis**

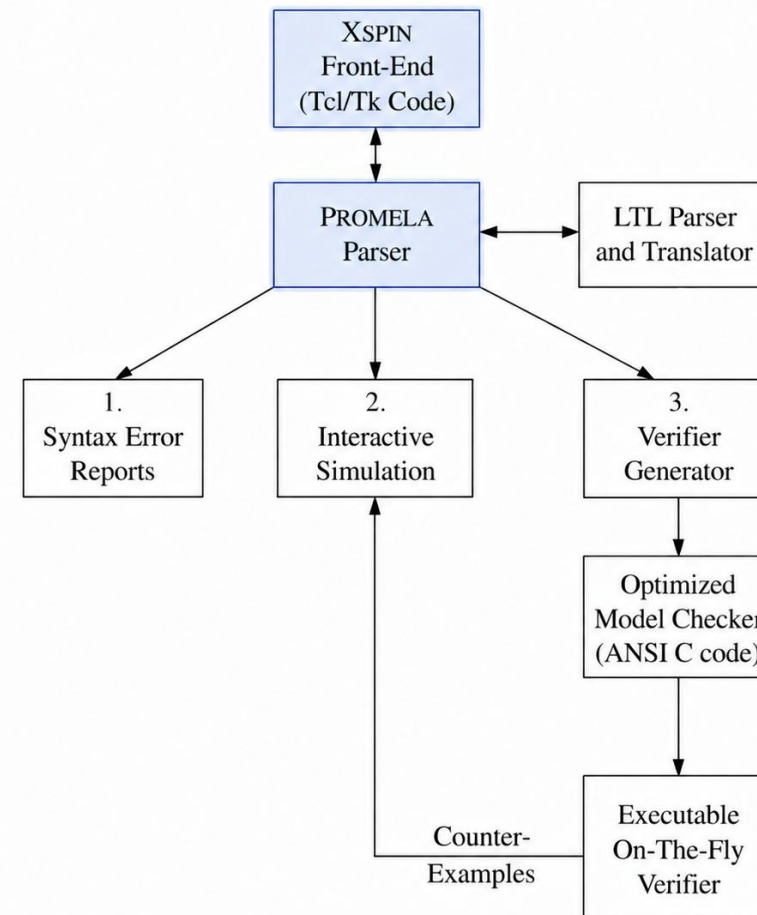


System Modeling with PROMELA

The first step is to identify the key properties of the design and build a **high-level model** of the concurrent system or distributed algorithm using the **PROMELA** specification language.

The system can be represented as a set of interacting **processes**, whose communication is modeled through **message channels**.

This phase is typically carried out through SPIN's graphical front-end, XSPIN.

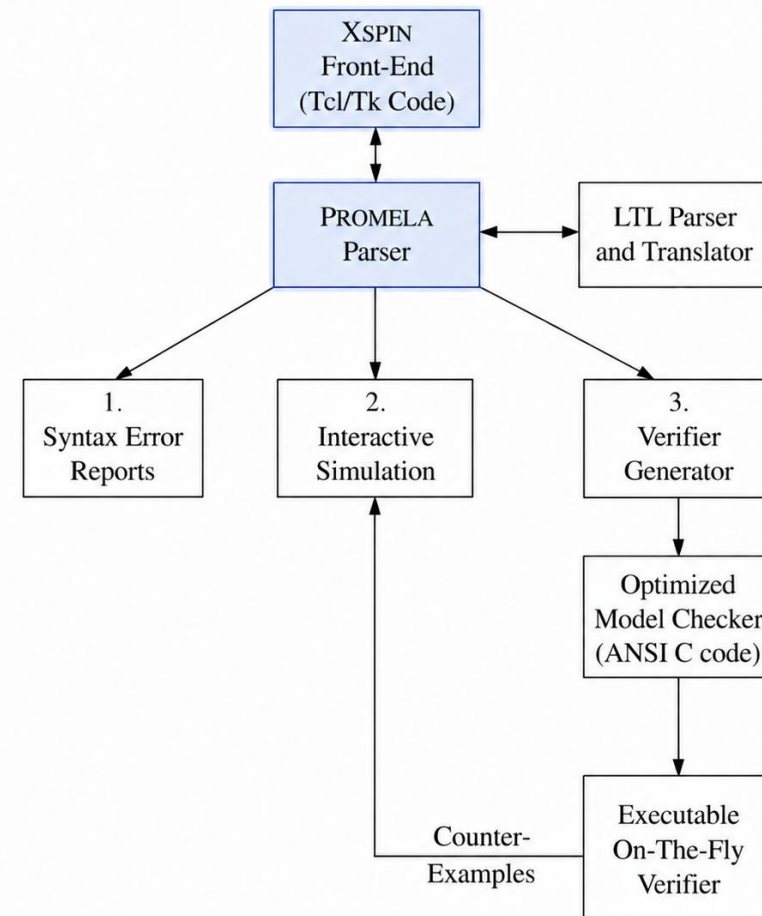


The resulting PROMELA code is then handled by the PROMELA parser, which processes the model and prepares it for subsequent analysis and verification.

Automata Generation

SPIN then translates each process into a finite automaton and computes the asynchronous interleaving product of these automata to obtain the global behavior of the concurrent system.

This interleaving product is often referred to as the **state space** of the system, and, because it can easily be represented as a graph, it is also commonly referred to as the global **reachability graph**.



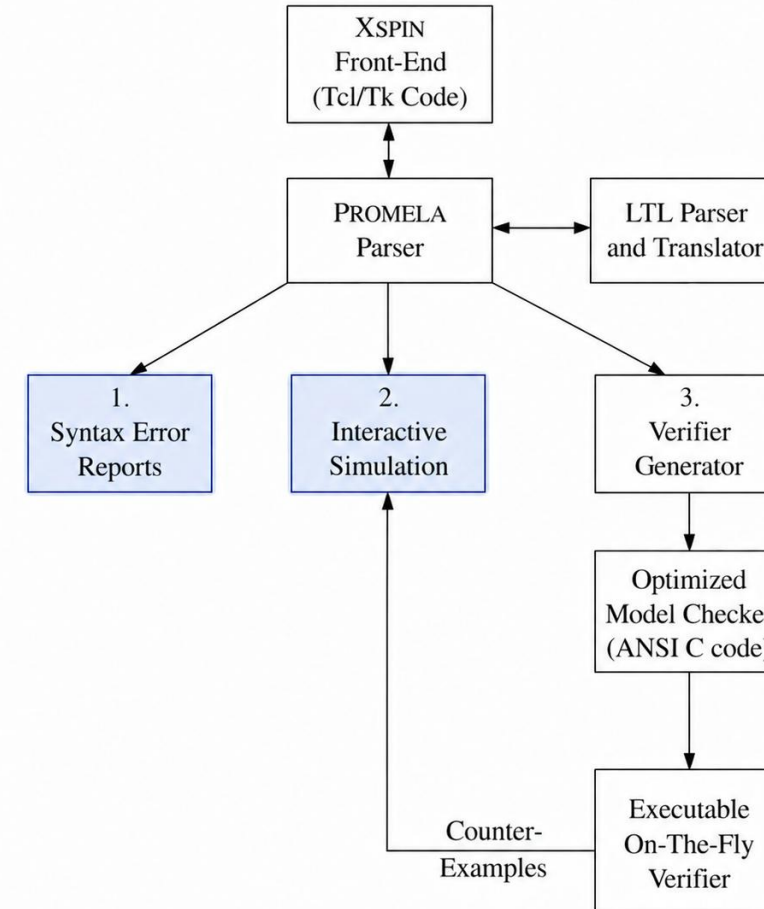
Syntax Check and Simulation

After drafting the model, SPIN (often via the XSPIN front-end) is used to identify and fix **syntax errors**.

After resolving errors, the model is tested using simulations, which can be:

- **Random simulation:** Automates non-deterministic choices via a random number generator for quick, approximate scenario testing.
- **Interactive simulation:** Since it is not always desirable to have SPIN automatically resolve all non-deterministic choices, this mode requires user intervention to manually resolve them.

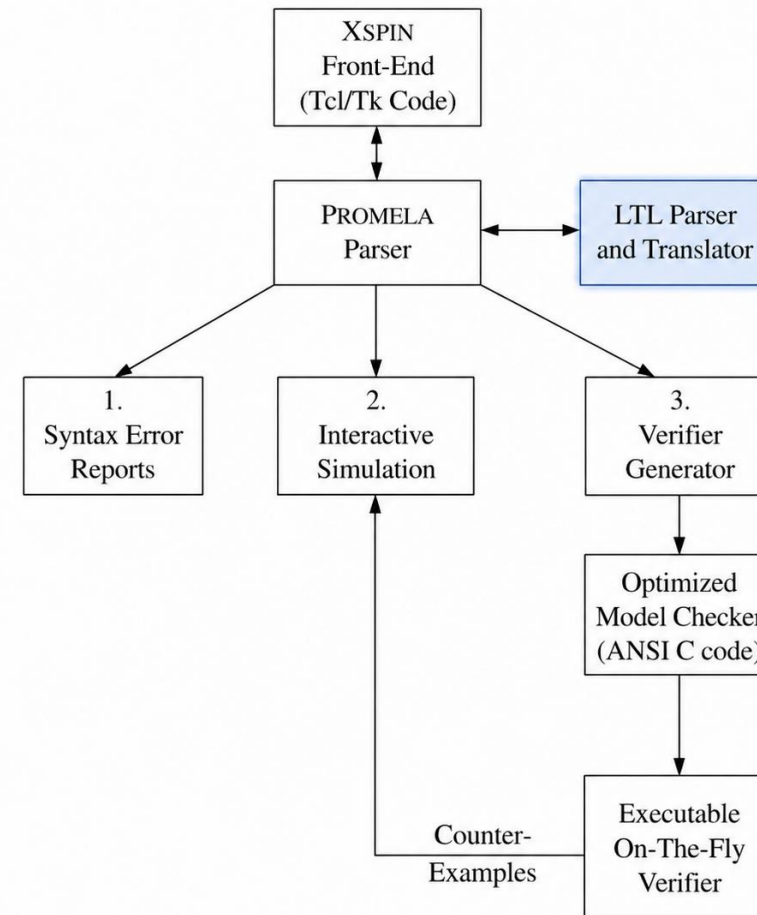
These simulations check approximate functionality and build **basic confidence**, rather than perform formal verification. System behaviour in these runs can be easily tracked and debugged using simple **print** statements or inline **assertions**.



Requirements Formalization

In this phase, the requirements that the system must satisfy are specified in Linear Temporal Logic (LTL) alongside the PROMELA code and are then processed by the LTL Parser and Translator.

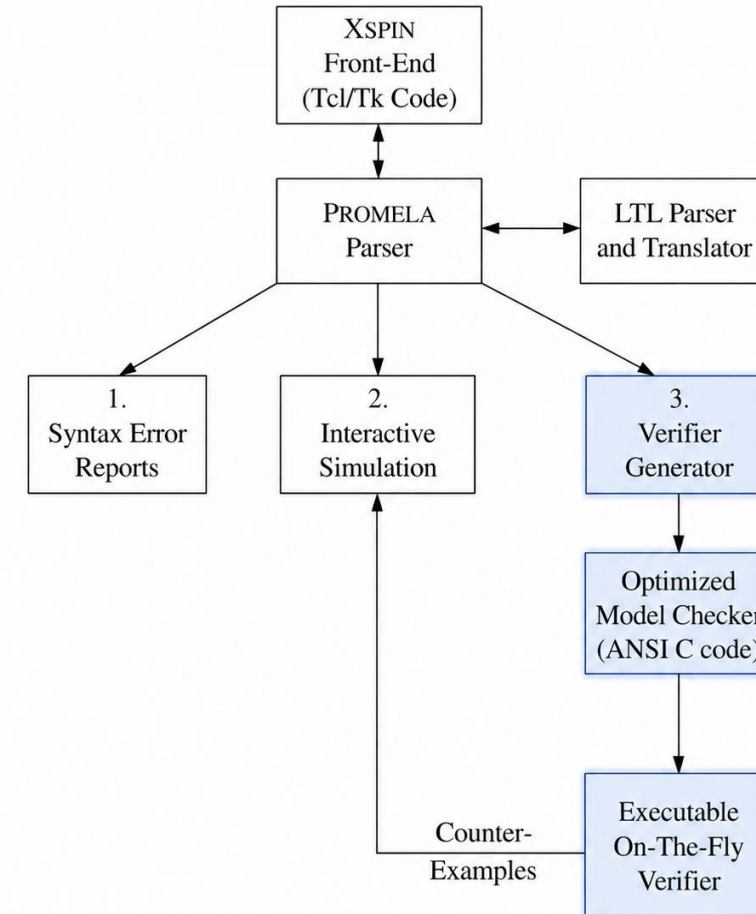
Once defined, the requirements are processed by SPIN's LTL parser, which mechanically translates each formula into a Büchi automaton. This automaton is then translated into PROMELA code as a specific construct known as a never claim.



Verifier Generation and Execution

In this execution step, the tool translates the high-level specification into a custom, highly optimized model checking program written entirely in **ANSI C**, rather than running the verification directly on the PROMELA code.

This generated **verification program** is then **compiled** by the user and executed. The resulting executable acts as an "on-the-fly" verifier, carrying out the verification during execution by dynamically analyzing the system's behaviour and halting as soon as a counterexample is found.



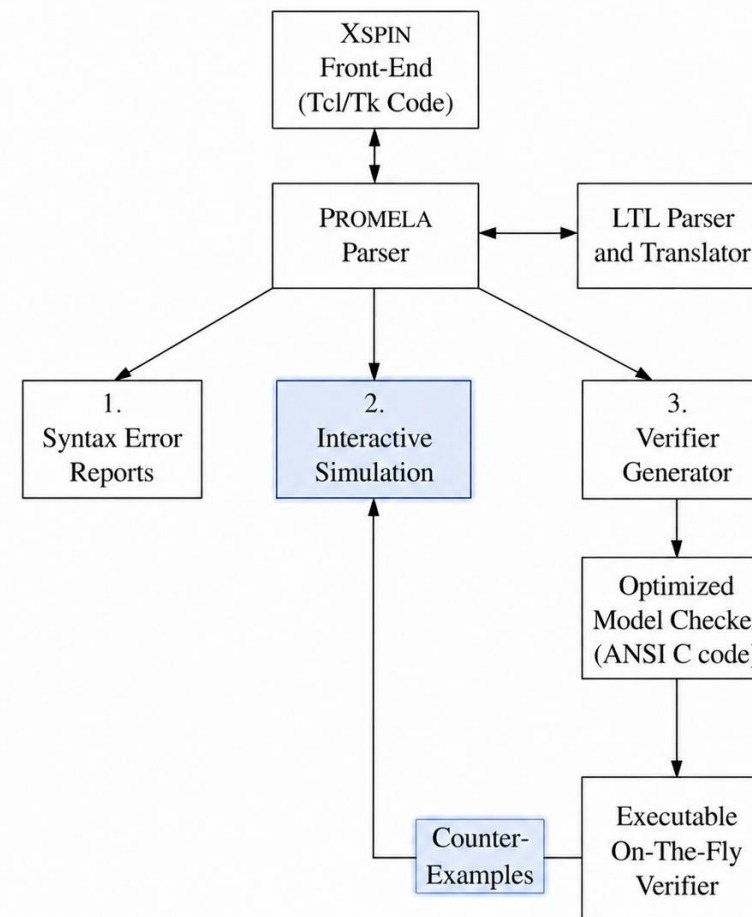
Counterexample Analysis

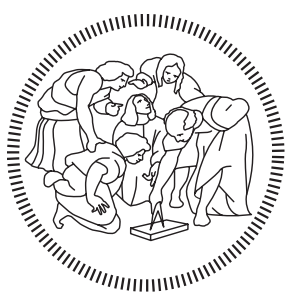
When the verifier detects a violation of a correctness property, it produces a **counterexample** in the form of a **sample execution**, a concrete sequence of system steps that leads to the error.

Guided Simulation

The counterexample execution is then **fed back** into the **simulator** and can be replayed through **guided simulation**, allowing step-by-step inspection of the exact scenario that caused the violation.

Based on this analysis, the model can be refined and the verification cycle repeated until all flaws are removed and correctness is achieved.





POLITECNICO
MILANO 1863

Büchi automaton in SPIN

Core idea

SPIN verifies temporal properties (such as safety and liveness) expressed in LTL by translating them into Büchi automaton.

This transformation turns a logical problem into an **automata-based problem**, which can be solved algorithmically.

In fact, it is known that every LTL formula can be translated into an equivalent Büchi automaton.

More in deep

In SPIN, the LTL formula is not used directly. Instead, it is negated and translated into a structure called a **never claim**, which is a Büchi automaton. This automaton represents **all behaviors that violate the property**.

Therefore, verification checks whether there exists an execution that satisfies the negation of the property:

- ➡ if such an execution exists, the **property is violated**;
- ➡ if no such execution exists, the **property holds**.

Büchi automaton in SPIN (2)

To perform verification, SPIN builds the **product** between:

- the system model
- the Büchi automaton of the negation of the LTL formula (**never claim**).

The resulting product automaton represents all possible executions of the system **constrained by the property**.

The verification problem is then reduced to detecting a specific pattern:

➡ an **accepting cycle**, that is, an infinite execution that visits accepting states infinitely often.

If such a cycle exists, the property is violated. In this case, SPIN is also able to provide a **counterexample execution**.

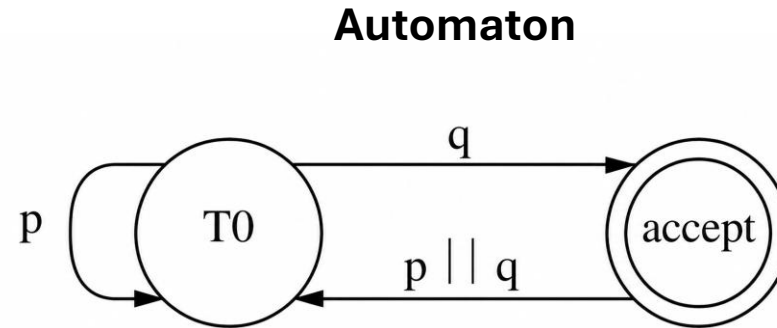
Otherwise, the property is guaranteed to hold for all executions of the system.

SPIN never builds the full product explicitly: it explores it **on-the-fly** and stops as soon as it finds an accepting cycle.

Büchi automaton in SPIN - Examples

First example

LTL Formula: $\Box (p \cup q)$
«it is always guarantee that
p remains true at least
until q becomes true»

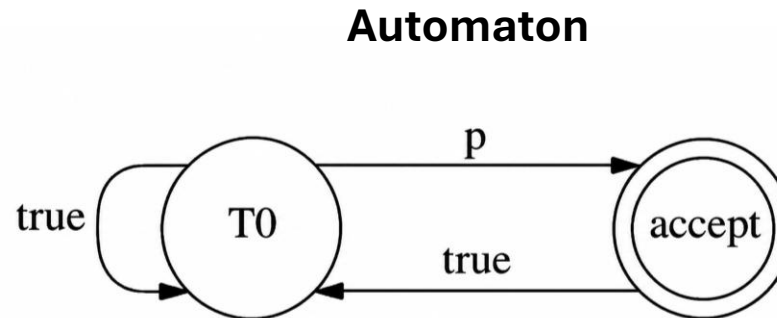


SPIN output

```
$ spin -f "[ (p U q) ]"
never { /* [ (p U q) */
T0_init:
    do
    :: ((q)) -> goto accept_S9
    :: ((p)) -> goto T0_init
    od;
accept_S9:
    do
    :: (((p)) || ((q)))) -> goto T0_init
    od;
}
```

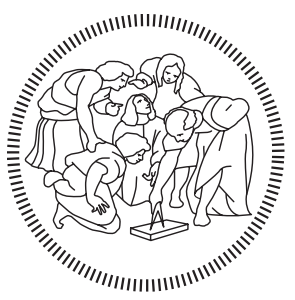
Second example

LTL Formula: $\Box (\Diamond p)$
«at any point in an
execution, it is guaranteed
that eventually p will
become true at least once
more»



SPIN output

```
$ spin -f "[ <Diamond> p ]"
never { /* [ <Diamond> p */
T0_init:
    do
    :: ((p)) -> goto accept_S9
    :: (1) -> goto T0_init
    od;
accept_S9:
    do
    :: (1) -> goto T0_init
    od;
}
```



POLITECNICO
MILANO 1863

Search algorithms

SPIN – High level overview

SPIN's primary verification engine — systematically explores the entire state space.

① Initialization

- Starts at the model's **initial state** (variables, process states)
- Initialize the **Stack/Queue** tracks the exploration path that **can be taken**
- **StateSpace** marks visited states to avoid revisiting

② Exploration

- **Select** an executable transition
- **Compute** the resulting new state
- **Add** new unvisited states to data structure and select a new executable transition
- **Backtrack** when no new transitions are available

③ What It Detects

- **Safety**
Assertion violations, deadlocks, invalid end states
 - **Liveness**
Acceptance cycles via LTL or never claims
- `./pan -E` `./pan -a`

Error Handling

When a violation is found, the search algorithm stops and outputs a **counterexample trail** (sequence of states/transitions).

Replay for debugging:
`spin -t -p file.pml`



SPIN – DFS – Cycle Detection

Extends DFS with a second search phase to detect acceptance cycles from accepting never state

Primary DFS Search

Standard DFS iteration over system, searching for **accepting never claim states**.

- Selects executable transitions, pushes unvisited states
- Backtracks when reaching visited state or dead-end
- When **backtracking from an accepting state** → launches Secondary DFS from that state
- When an **accepting state** (of $\neg P$) is found => launches **nested DFS** from that state

Nested DFS Search

Launched from the **seed** (accepting state) to find a path back to itself. Uses a separate stack + separate visit bits

- If **t == seed** is found → **acceptance cycle detected** => **liveness violation found**
- Only 2 bits per state added making the algorithm space-efficient but verification time may double if no cycle exists

Complete DFS Search Pseudocode

```
proc dfs(s)
  if error(s) then report error fi
  add s to Statespace
  for each successor t of s do
    if accepting(s) then seed:=s; ndfs(s) fi
    if t not in Statespace then dfs(t) fi
  od
end
```

```
proc ndfs(s)
  add {s,1} to Statespace
  for each successor t of s do
    if {t,1} not in Statespace then ndfs(t) fi
    else if t==seed then report cycle fi
  od
end
```

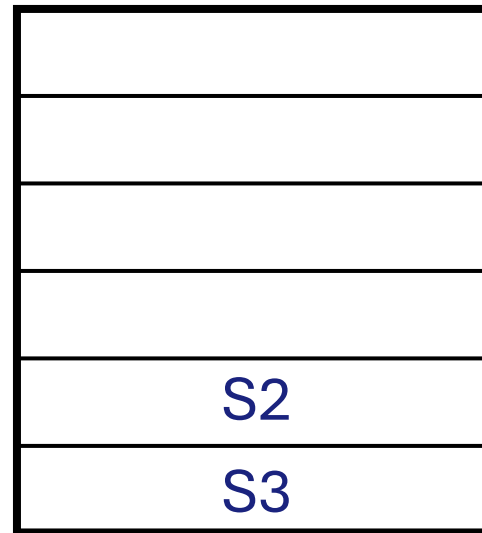
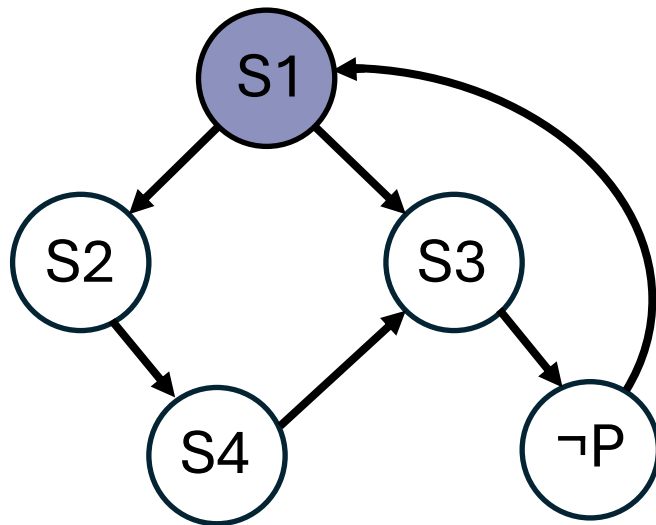


SPIN – DFS – Simulation 1

Complete DFS Search Pseudocode

```
proc dfs(s)
  if error(s) then report error fi
  add s to Statespace
  for each successor t of s do
    if accepting(s) then seed:=s; ndfs(s) fi
    if t not in Statespace then dfs(t) fi
  od
end
```

```
proc ndfs(s)
  add {s,1} to Statespace
  for each successor t of s do
    if {t,1} not in Statespace then ndfs(t) fi
    else if t==seed then report cycle fi
  od
end
```



Stack



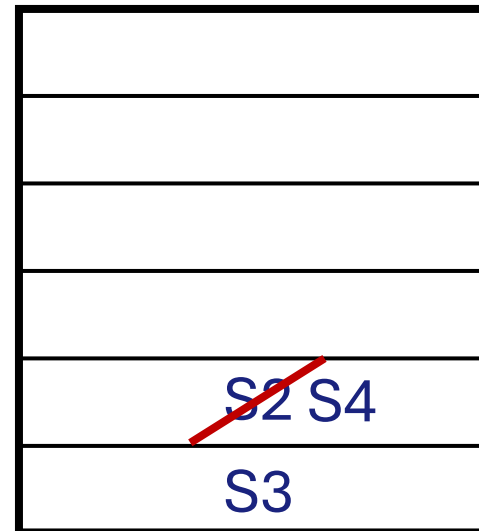
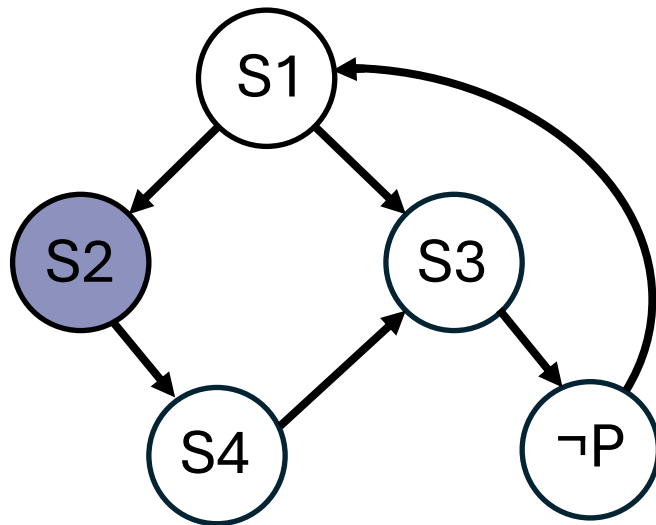
StateSpace

SPIN – DFS – Simulation 2

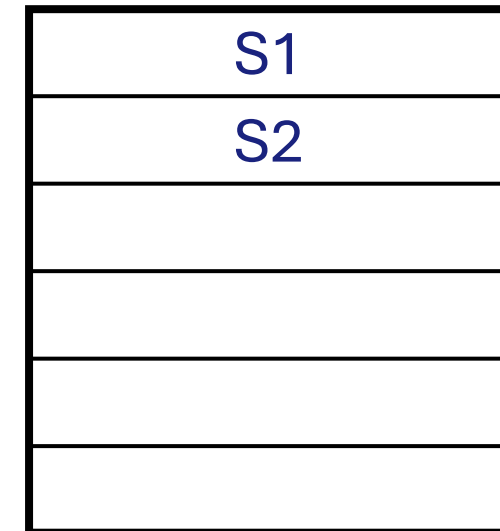
Complete DFS Search Pseudocode

```
proc dfs(s)
  if error(s) then report error fi
  add s to Statespace
  for each successor t of s do
    if accepting(s) then seed:=s; ndfs(s) fi
    if t not in Statespace then dfs(t) fi
  od
end
```

```
proc ndfs(s)
  add {s,1} to Statespace
  for each successor t of s do
    if {t,1} not in Statespace then ndfs(t) fi
    else if t==seed then report cycle fi
  od
end
```



Stack



StateSpace

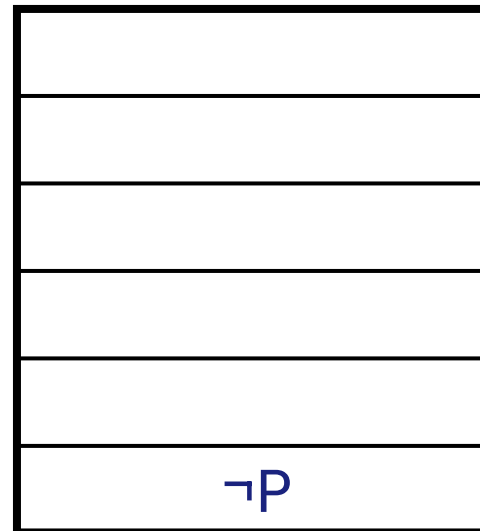
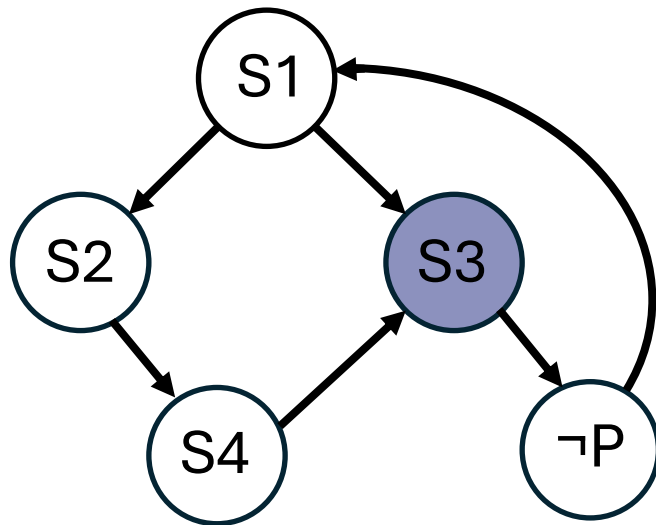


SPIN – DFS – Simulation 4

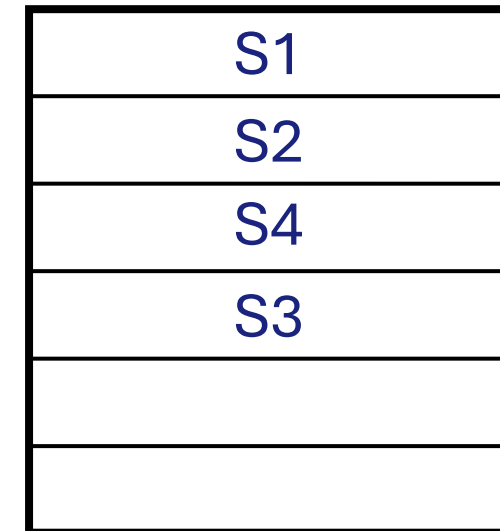
Complete DFS Search Pseudocode

```
proc dfs(s)
  if error(s) then report error fi
  add s to Statespace
  for each successor t of s do
    if accepting(s) then seed:=s; ndfs(s) fi
    if t not in Statespace then dfs(t) fi
  od
end
```

```
proc ndfs(s)
  add {s,1} to Statespace
  for each successor t of s do
    if {t,1} not in Statespace then ndfs(t) fi
    else if t==seed then report cycle fi
  od
end
```



Stack



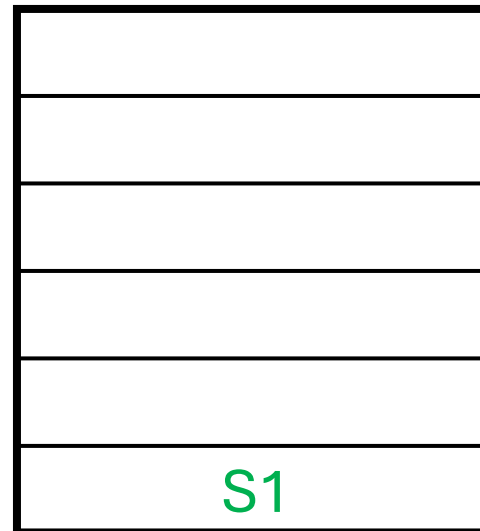
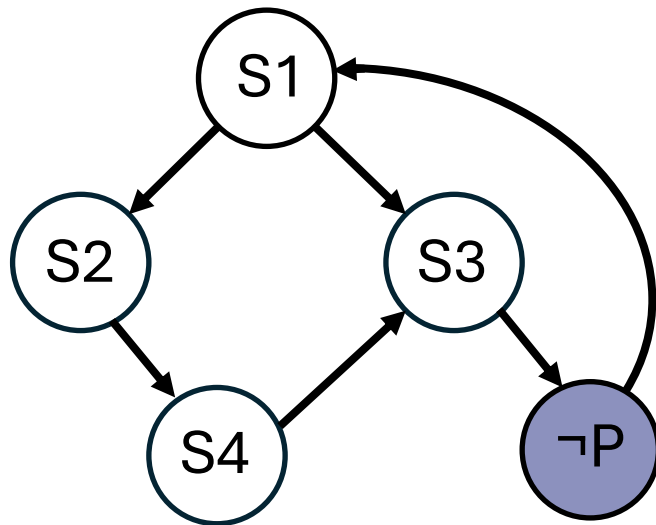
StateSpace

SPIN – DFS – Simulation 5

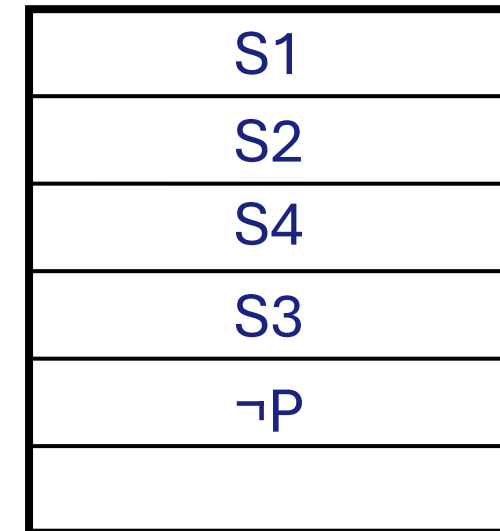
Complete DFS Search Pseudocode

```
proc dfs(s)
  if error(s) then report error fi
  add s to Statespace
  for each successor t of s do
    if accepting(s) then seed:=s; ndfs(s) fi
    if t not in Statespace then dfs(t) fi
  od
end
```

```
proc ndfs(s)
  add {s,1} to Statespace
  for each successor t of s do
    if {t,1} not in Statespace then ndfs(t) fi
    else if t==seed then report cycle fi
  od
end
```



Stack



StateSpace

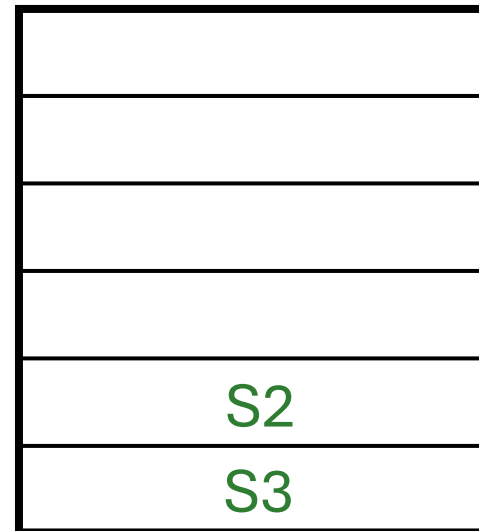
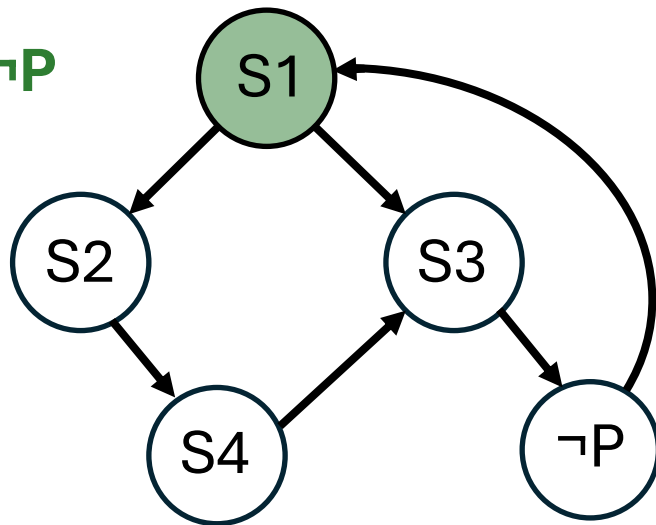
SPIN – DFS – Simulation 6

Complete DFS Search Pseudocode

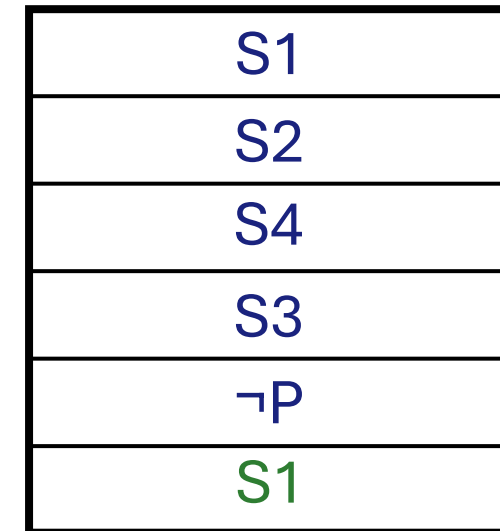
```
proc dfs(s)
  if error(s) then report error fi
  add s to Statespace
  for each successor t of s do
    if accepting(s) then seed:=s; ndfs(s) fi
    if t not in Statespace then dfs(t) fi
  od
end
```

```
proc ndfs(s)
  add {s,1} to Statespace
  for each successor t of s do
    if {t,1} not in Statespace then ndfs(t) fi
    else if t==seed then report cycle fi
  od
end
```

SEED = $\neg P$
T = S1



Stack



StateSpace





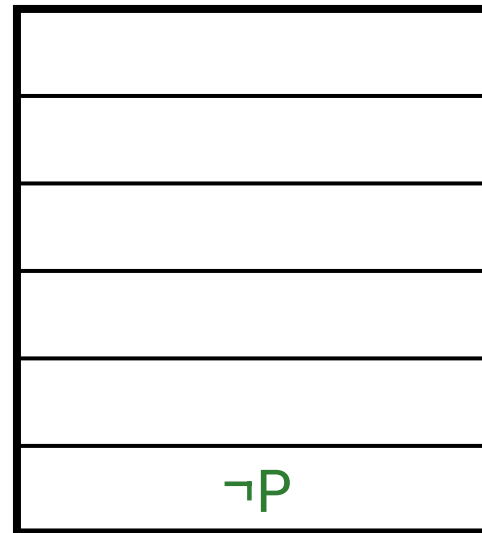
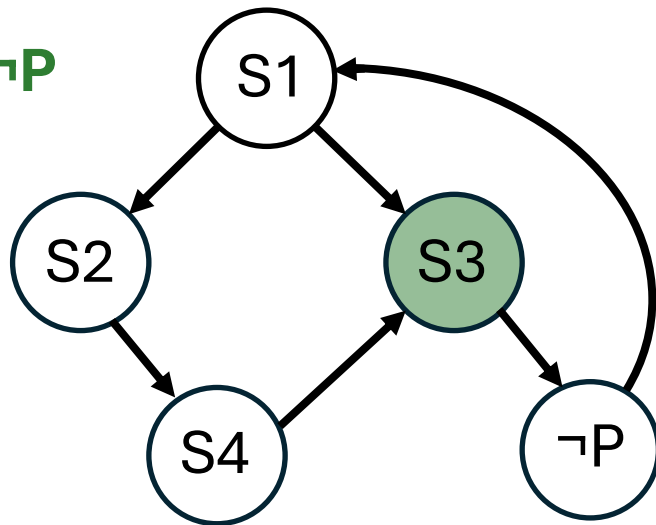
SPIN – DFS – Simulation 9

Complete DFS Search Pseudocode

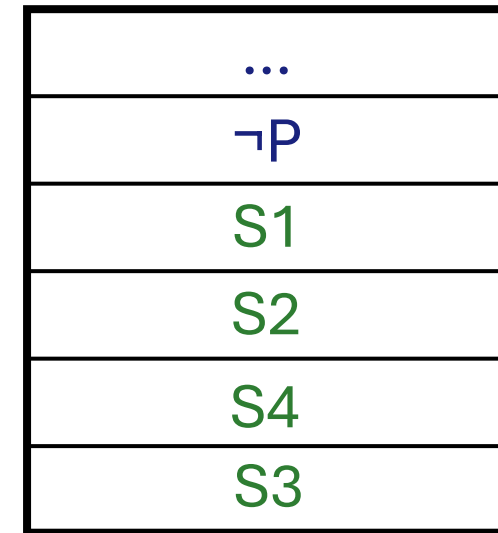
```
proc dfs(s)
  if error(s) then report error fi
  add s to Statespace
  for each successor t of s do
    if accepting(s) then seed:=s; ndfs(s) fi
    if t not in Statespace then dfs(t) fi
  od
end
```

```
proc ndfs(s)
  add {s,1} to Statespace
  for each successor t of s do
    if {t,1} not in Statespace then ndfs(t) fi
    else if t==seed then report cycle fi
  od
end
```

SEED = $\neg P$
T = S1



Stack



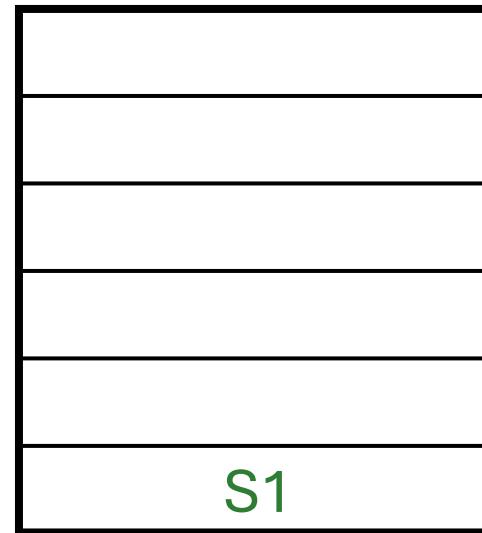
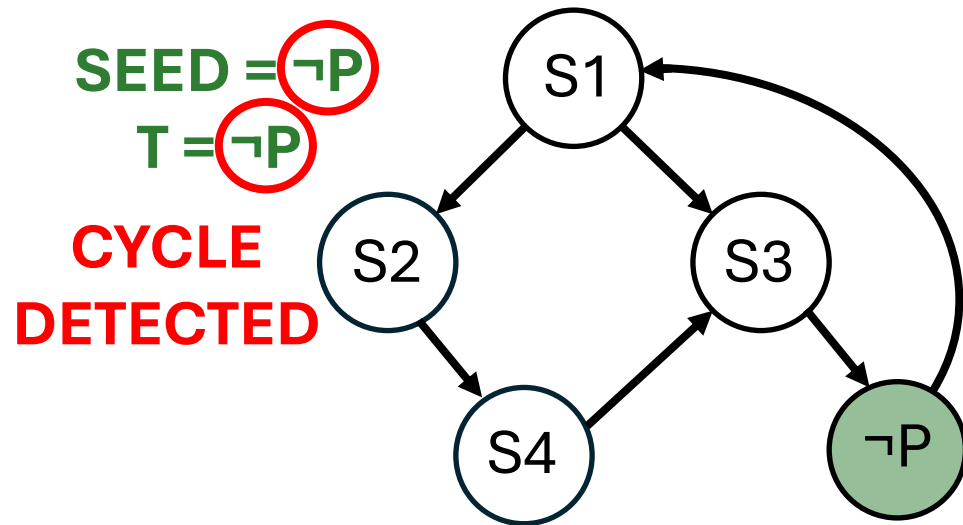
StateSpace

SPIN – DFS – Simulation 10

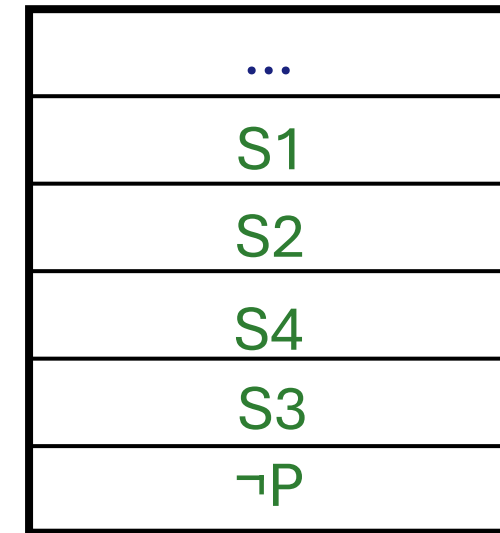
Complete DFS Search Pseudocode

```
proc dfs(s)
  if error(s) then report error fi
  add s to Statespace
  for each successor t of s do
    if accepting(s) then seed:=s; ndfs(s) fi
    if t not in Statespace then dfs(t) fi
  od
end
```

```
proc ndfs(s)
  add {s,1} to Statespace
  for each successor t of s do
    if {t,1} not in Statespace then ndfs(t) fi
    else if t==seed then report cycle fi
  od
end
```



Stack



StateSpace



SPIN – Breadth-First Search – BFS

Explores state space level-by-level and enable spin to find shorter counterexample paths.

How BFS Works

Data structure: Queue (FIFO) instead of Stack

1. Enqueue initial state; mark as visited
2. Dequeue state s from front of queue
3. For each executable transition $\rightarrow$ compute s'
If unvisited: mark + enqueue to back
4. Repeat until queue empty or error found

Enable with: `gcc -DBFS -o pan pan.c`

BFS Pseudocode

```
Queue D = {}
Statespace V = {}
Start() {
  Add_Statespace(V, A.s0)
  Add_Queue(D, A.s0)
  Search()
}
Search() {
  s = Del_Queue(D)
  for each (s,l,s') in A.T
    if not In_Statespace(V,s')
      { Add_Statespace(V, s')
        Add_Queue(D, s')
        Search() }
}
```

DFS vs BFS

Data structure	Stack (LIFO)	Queue (FIFO)
Memory use	Low — only current path	High — all frontier states
Best for	Deep errors, liveness (cycles)	Shallow errors, short counter example



SPIN – Search Algorithm – Optimizations - 1

Techniques to tackle the state explosion problem in order to reduce memory and time cost without sacrificing correctness

Bounded Context Switching

Bounded Context Switching **reduces the state space by restricting exploration to executions with at most K process switches.**

For each state, SPIN records the minimal number of switches needed to reach it **and discards paths that reach the same state with more switches.** This focuses the search on simpler interleavings, which are often sufficient to expose concurrency bugs.

Collapse Compression

Splits global state descriptors (variables, channel ecc) into components, assign a unique indices and store them in tables.

This allow to reduce the memory occupied by a single state if it is composed of many shared components.

While reducing per-state memory usage, it increase verification time. Can be enable by the -DCOLLAPSE flag

Minimal DFA

Reduces memory usage by storing reachable states in a minimal DFA instead of a lookup table.

If two state are describe by the same descriptor, only one of them is present in the DFA instead of having multiple copy in a look up table

Bitstate Hashing

Bitstate Hashing uses a large bit array and multiple hash functions to record visited states. Instead of storing full state descriptors, **each state sets one or more bits in the array. When visiting a new state, SPIN checks these bits to determine if it was likely seen before.** This approach is highly memory-efficient but lossy, as hash collisions can cause states to be missed. It can be enabled with the -DBITSTATE flag.



SPIN – Search Algorithm – Optimizations - 2

Techniques to tackle the state explosion problem in order to reduce memory and time cost without sacrificing correctness

Partial Order Reduction

Reduces the number states explored in SPIN verification, minimizing state space size.

Instead of exploring all of the possible path order only dependent action.

Based on static technique, the optimization happens at compile time.
It's enabled by default in spin

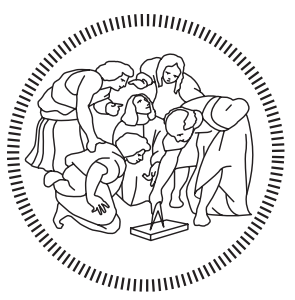
Hash-Compact

Hash-Compact **reduces memory usage by storing a fixed-size hash signature of each state** instead of the full state descriptor.

This allows SPIN to keep many more states in memory, at the cost of possible hash collisions, which may cause different states to be treated as identical.

Unlike Bitstate Hashing, it stores one compact fingerprint per state rather than using a bit array, making it less lossy and closer to standard exhaustive search.





POLITECNICO
MILANO 1863

PROMELA language

PROMELA - Processes

proctype

Declares a process behavior but does **not** start it

```
proctype Pname(optional argument) {  
    // logic to execute  
}
```

Each instance gets its own `_pid` (process counter)

active [n]

Starts automatically when model begins.
[n] spawns n copies.

```
active [3] proctype P() {  
    // logic to execute spawn 3  
    copies  
}
```

If [n] is omitted, n is implied to be 1

run

Dynamically **starts a process from inside another process.**

```
proctype P() {  
    //code  
}  
  
active P1() {  
    run P();  
}
```

init

Special process that runs automatically at model startup, only one exists

```
init {  
    run Pname();  
}
```

The first line in init is always executed before anything else

Scheduling

The PROMELA language is built around the concept of **non-determinism**. This makes the execution flow very different from other standard languages. In particular:

- The **first line** of *init* is always **executed first**
- Then all processes are scheduled **non-deterministically**.

Use the **atomic{}** or **d_step{}** keyword to create an **atomic and not interleaved** piece of code



```
proctype P1() {
  printf("P1\n");
}

active P2() {
  printf("P2\n");
}

active P3() {
  atomic{
    printf("P3\n")
    printf("P3 copy\n")
  }
}

init {
  print("Init\n")
  run P1();
}
```

Running this test can lead to different output:

- **Init, P1, P3, P3 copy, P2**
- **Init, P1, P2, P3, P3 copy**
- **Init, P2, P1, P3, P3 copy**
- **Init, P2, P3, P3 copy, P1**
- **Init, P3, P3 copy, P2, P1**
- **Init, P3, P3 copy, P2, P1**

Notice how the *Init print* is always executed first.

The *P3* and *P3 copy print* are always printed sequentially since they are inside an *atomic block*

PROMELA - Variables & Data Types

TypeRange / Values	
bit	0, 1
bool	false, true
byte	0 .. 255
short	$-2^{15} .. 2^{15}-1$
int	$-2^{31} .. 2^{31}-1$
unsigned	$0 .. 2^n-1$
chan	channel handle (1..255)
mtype	named constants (1..255)

Key Rules

- All variables default to 0
- No floats or no strings supported
- **Scope:** variable can either be **local** or **global**

mtype: Named Constants

Define symbolic names, is equivalent to an *enum* in Java

```
mtype = { RED, GREEN, BLUE };  
mtype color = GREEN;
```

chan: Channel Handle

Channels are used to communicate between process, it represent a message queue:

```
chan pipe = [4] of { byte, bool };
```



PROMELA - Control Flow

Promela control semantics is based on Dijkstra's Guarded Commands but with a C-like notation

Selection (if)

Once all guards have been evaluated **one true branch is chosen non-deterministically**. Blocks the process if all guard are false.

Repetition (do)

Allow to create loop in the PROMELA language. Can be escaped by using the **unless** construct

Unless

Escape block: if any of his guard becomes true, interrupts the main body immediately. Act like a control variable

Jump (goto)

Unconditional **jump to a labeled statement** within the same scope

The Power of Non-Determinism

When **multiple guards** in an if or do **are simultaneously true**, **SPIN explores ALL possible choices**. This is what makes it a model checker, not a simulator. A **process blocks** if no guard is true, waiting for a concurrent process to change the state.



PROMELA – Selection (if)

Syntax

```
if
:: guard1 -> action1
:: guard2 -> action2
:: else   -> action3
fi
```

guard = the condition before the arrow that needs to be true to execute it

Execution Rules

- **One branch executes**

Exactly **one sequence** from the list runs **per if evaluation**.

- **Non-deterministic choice**

If **multiple guards are true** simultaneously, **one** is **picked at random**

- **Blocking**

If **no guard is true**, the **process blocks** until a concurrent process makes one true.

- **Else clause**

Only fires if all other guards are false. Provides a way to not block process execution

Example: Non-Deterministic Max

```
if
:: a >= b -> max = a    // branch 1 taken if a >= b
:: b >= a -> max = b    // branch 2 taken if b >= a
fi                      // when a==b, SPIN explores BOTH
```

When $a == b$ both guards are true → SPIN explores both branches to verify all possible outcomes.

Both guards can be taken!



PROMELA – Repetition (do), Unless & Goto

do: Loop with Guard-Driven Exit

```
do
:: condition1 -> break
:: //code to be repeated
od
```

Do-While pattern

break exits the loop immediately.

If a more complex exit logic is needed, more guard can be put with break, simulating a OR exit condition

do: For loop

```
#define N 5
byte i = 1; int sum = 0;
do
:: i > N -> break
:: else -> sum = sum + i; i++
od
```

For-loop pattern

unless : Interruption / Escape

Monitors an escape condition **in parallel**.

If the guard fires, main body is abandoned.

```
do
:: i <= 10 ->
    printf("Counting: %d\n", i);
    i++
od
unless
:: pressed ->
    printf("Interrupted!\n")
```

goto: Jump

Allows jumping to any point within the same scope. Excessive use may reduce code readability and maintainability.

```
do
:: i > N -> goto exitloop
:: else ->
od
exitloop
```



PROMELA – Channel Communication

Channels are First-In-First-Out (FIFO) message queues. It's the primary method used by process to communicate

Declaration

```
chan chname = [capacity] of { type1, type2, ... };
```

Buffered (capacity > 0)

```
chan buf = [8] of { byte };
buf!42;    // send 42 over buf
buf?val;   // receive data from buf, save it in val
```

Sender and receiver can be *asynchronous*.

Rendezvous (capacity = 0)

```
chan sync = [0] of { byte };
sync!msg;  // blocks execution until receiver ready
sync?val;  // blocks execution until sender ready
```

Sender and receiver *synchronize* at message exchange.

Primitives

```
ch!val1, val2 // send (multiple fields)
ch?var1, var2 // receive into variables
```

Status Built-ins

len(ch): number of msgs in queue
empty **nempty**: empty or not-empty, bool



PROMELA – Model Verification

assert: Safety Checks

Halts verification and reports a counterexample if this condition is violated.

```
assert(x >= 0);    // stops if x < 0
```

progress / accept — Liveness

Labels placed on states to verify liveness — e.g. a system must eventually make progress and never stall in an accept cycle.

LTL Formulas and Temporal Properties

Express properties that must hold over time across all execution paths, not just at one instant.

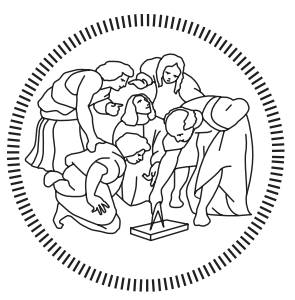
```
spin -f "[ ] (x <= 5)"    →    generates never { ... } block
```

LTL Quick Reference

[]p	Always	p holds in every future state
<>p	Eventually	p will be true at some future point
p U q	Until	p holds continuously until q becomes true
p V q	Release	q holds until (and including when) p becomes true
p -> q	Implies	whenever p is true, q must also be true

Workflow: `spin -a model.pml` → `gcc -o pan pan.c` → `./pan -n`





POLITECNICO
MILANO 1863

Tool setup

Setup: Windows

Setup on Windows

- 1) Download SPIN suitable binary  [Spin/Bin folder of Github repository](#) and extract the executable file
- 2) Download Cygwin  [Cygwin website](#) making sure to install the following packages during setup: **bison**, **gcc-core**, **graphviz**, **make**, **tcl** and **tcl-tk**
- 3) Rename the previously downloaded executable to “**spin.exe**” and place it in the “**cygwin/bin**” directory.
- 4) Add “*/cygwin/bin*” to the system PATH environment variable
- 5) To use SPIN: open the **Cygwin Terminal**, go to the folder containing the *.pml* file, and run SPIN commands

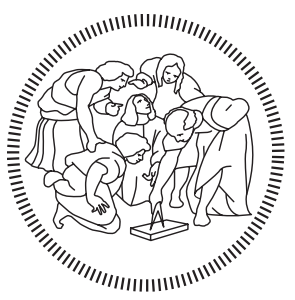


Setup: Linux/WSL (Ubuntu)

Setup on Linux

- 1) Install the required packages (dependencies) ➡ `sudo apt install build-essential gcc`
- 2) Clone the [SPIN repository](https://github.com/nimble-code/Spin.git) from GitHub ➡ `git clone https://github.com/nimble-code/Spin.git`
- 3) Move to the *Spin*/Src folder ➡ `cd <clone-path>/Spin/Src`
- 4) Build SPIN by running the command *make*
- 5) Add the SPIN installation path to `~/.bashrc`:
 - 1) open the file with `nano ~/.bashrc`
 - 2) add `export PATH="$PATH:/path/to/your/spin_folder"` at the end of the file
 - 3) apply the changes with `source ~/.bashrc`
- 6) To use SPIN: open the **terminal**, go to the folder containing the *.pml* file, and run SPIN commands





POLITECNICO
MILANO 1863

Extensions

Purpose

iSPIN is the graphical user interface for SPIN. It is the successor to XSPIN. It helps users edit PROMELA models, run simulations, launch verifications, inspect results, and visualize process automata.

How to install and open iSPIN (assuming SPIN is already installed, as explained in the previous slides)

Windows

- **Install** a native Windows Tcl/Tk distribution from [MagicSplat](#) (use version 8.6.x)
- **Get** ispin.tcl from the [SPIN repository](#)
- **Launch** iSPIN from the **Cygwin terminal** with:
`"/cygdrive/c/.../wish.exe" "/cygdrive/.../ispin.tcl"`

Linux

- **Install** Tcl/Tk using your distribution package manager (e.g., apt, df, pacman). Make sure a compatible Tcl/Tk 8.x version is available (iSPIN may not work with Tcl/Tk 9)
- **Launch** iSPIN from *Spin/optional_gui* folder with:
`wish8.6 ./ispin.tcl`

iSPIN Overview

Spin Version 6.5.1 -- 20 December 2019 :: iSpin Version 1.1.5 -- 28 May 2021

Edit/View Simulate / Replay Verification Swarm Run <Help> Save Session Restore Session <Quit>

Open... ReOpen Save Save As... Syntax Check Redundancy Check Symbol Table Find:

```
1 #define N 5 /* Five philosophers */
2
3 bool stick[N];
4
5 inline take(s) {
6     atomic {
7         stick[s] == false ->
8         stick[s] = true
9     }
10 }
11
12 inline release(s) {
13     atomic {
14         stick[s] = false
15     }
16 }
17
18 active [N] proctype Philosopher() {
19     byte id = _pid; /* id of the philosopher */
20     byte left = id; /* id of the left stick */
21     byte right = (id + N - 1) % N; /* id of the right stick */
22
23     do
24     ::
25         if
26         :: take(left);
27         :: take(right);
28         :: take(right);
29         :: take(left);
30         fi;
31
32         /* eating */
33         skip;
34 }
35
```

Automata View zoom in zoom out

Select: p_Philosopher

stick[left] = 0

Spin Version 6.5.1 -- 20 December 2019
iSpin Version 1.1.5 -- 28 May 2021
Tcl/Tk Version 8.6/8.6
1 C:/Users/Matteo/Desktop/demo_spin/good_liveness/good_liveness.pml:1
2 C:/Users/Matteo/Desktop/demo_spin/bad/bad.pml:1
3 spin -o3 -a bad.pml
4 gcc -o pan pan.c
5 ./pan -D > dot.tmp
6 select p_Philosopher
7 verification
8 verification

Purpose

Swarm is a parallel search extension for SPIN, that runs many diversified verification processes in parallel. It is used to find bugs faster in very **large models**, when exhaustive verification is too expensive. It can also be used through the dedicated “Swarm Run” section in iSPIN.

How to install and open Swarm (assuming SPIN is already installed, as explained in the previous slides)

Windows

- Download (or clone) the [Swarm repository](#) and put its contents into the Cygwin directory
- From the Cygwin terminal, run the command *make* in the *Swarm/src* directory
- Move the generated “*swarm.exe*” file into *cygwin/bin* directory
- Open iSPIN and use the “*Swarm Run*” section

Linux

- Clone the [Swarm repository](#)
- Run the command *make* in *Swarm/Src* directory
- Copy the swarm executable file into “*/usr/bin*” with “*cp swarm /usr/bin/*” command
- Open iSPIN and use the “*Swarm Run*” section

Swarm Overview

Search Constraints		Estimates (Fine Tuning)		Compilation Options (any number, one per line)	
minimum nr of hash functions:	1	hash-factor	1.5	-DBITSTATE -DPUTPID	# basic dfs
maximum nr of hash functions:	5	state-vector size in bytes	512	-DBITSTATE -DPUTPID -DREVERSE	# reversed transition ordering
minimum search depth:	100	exploration speed in states/sec	250000	-DBITSTATE -DPUTPID -DT_REVERSE	# reversed process ordering
maximum search depth:	10000			-DBITSTATE -DPUTPID -DREVERSE -DT_REVERSE	# both
number of local cpu-cores	4	Model Generation		-DBITSTATE -DPUTPID -DP_RAND -DT_RAND	# same series with randomization
list of remote_cpu_names:cores		extra spin args		-DBITSTATE -DPUTPID -DP_RAND -DT_RAND -DT_REVERSE	
maximum memory per run (suffic M or G)	512M	extra pan args	-c1 -x -n -a	-DBITSTATE -DPUTPID -DP_RAND -DT_RAND -DREVERSE	
maximum total runtime for swarm (suffic s, m, h, d)	60m			-DBITSTATE -DPUTPID -DP_RAND -DT_RAND -DREVERSE -DT_REVERSE	

Run Cleanup tmp files

warning: never claim + accept labels requires -a flag to fully verify
 warning: never claim + accept labels requires -a flag to fully verify
 generated configuration file
 done:: swarm: reading settings from 'swarm_cfg.tmp'
 swarm: 49 runs, avg time per cpu 3598.7 sec
 swarm: script written to good_liveness.pml.swarm
 ----Running----
 !t starvation_free_all: [] ((((((! (hungry[0])) || (<> (eating[0])))) && (! (hungry[1])) || (<> (eating[1]))))) && (! (hungry[2])) || (<> (eating[2])))) && (! (hungry[3])) || (<> (eating[3])))) && (! (hungry[4])) || (<> (eating[4]))))
 run completed

```

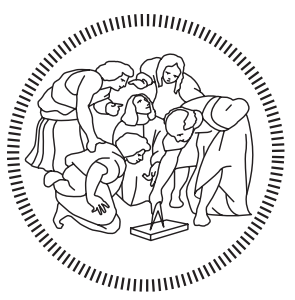
script1.out:State-vector 88 byte, depth reached 9999, errors: 0
script1.out:State-vector 88 byte, depth reached 9116, errors: 0
script1.out:State-vector 88 byte, depth reached 8454, errors: 0
script2.out:State-vector 88 byte, depth reached 9500, errors: 0
script2.out:State-vector 88 byte, depth reached 9999, errors: 0
script2.out:State-vector 88 byte, depth reached 127, errors: 0
script2.out:State-vector 88 byte, depth reached 127, errors: 0
script2.out:State-vector 88 byte, depth reached 9720, errors: 0
script2.out:State-vector 88 byte, depth reached 9999, errors: 0
script2.out:State-vector 88 byte, depth reached 9586, errors: 0
script2.out:State-vector 88 byte, depth reached 9999, errors: 0
script2.out:State-vector 88 byte, depth reached 9644, errors: 0
script2.out:State-vector 88 byte, depth reached 9999, errors: 0
script2.out:State-vector 88 byte, depth reached 8952, errors: 0
script2.out:State-vector 88 byte, depth reached 9414, errors: 0
script3.out:State-vector 88 byte, depth reached 9752, errors: 0
script3.out:State-vector 88 byte, depth reached 9999, errors: 0
script3.out:State-vector 88 byte, depth reached 127, errors: 0
script3.out:State-vector 88 byte, depth reached 127, errors: 0
script3.out:State-vector 88 byte, depth reached 9999, errors: 0
script3.out:State-vector 88 byte, depth reached 9999, errors: 0
script3.out:State-vector 88 byte, depth reached 9600, errors: 0
script3.out:State-vector 88 byte, depth reached 9999, errors: 0
script3.out:State-vector 88 byte, depth reached 9492, errors: 0
script3.out:State-vector 88 byte, depth reached 9999, errors: 0
script3.out:State-vector 88 byte, depth reached 8296, errors: 0
script3.out:State-vector 88 byte, depth reached 7854, errors: 0

/usr/bin/ls: cannot access '*.trail': No such file or directory
  
```



Other extensions

- Modex** → extracts PROMELA models from C code for software verification.
- jSPIN** → Java-based graphical interface for SPIN.
- XSPIN** → legacy graphical front-end for SPIN, used to edit PROMELA models, run simulations, launch verifications, and inspect counterexamples.
- Real-time SPIN extension** → adds timing features for the verification of real-time systems.



POLITECNICO
MILANO 1863

Demo preview

Dining philosophers in SPIN – Deadlock detection

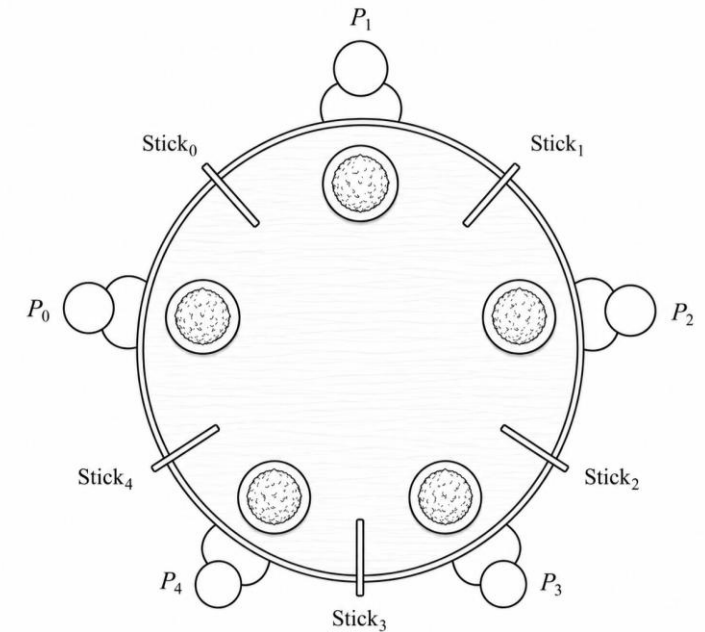
A set of N philosophers sit around a table and share N chopsticks. Each philosopher needs the two adjacent sticks to eat.

Initially, each philosopher may nondeterministically choose which stick to pick up first. If every philosopher picks up one stick and waits for the other, a **deadlock** may occur.

We will model this scenario in SPIN and automatically check if such a deadlock can arise, considering just five philosophers and sticks.

To avoid deadlock, we remove the arbitrary choice of the first stick and break the symmetry: philosopher P_0 picks up the right stick first, while all the others pick up the left stick first.

This prevents circular waiting and ensures that deadlock cannot occur.



Dining philosophers in SPIN – Starvation freedom

In addition to deadlock detection, we verify a liveness property using LTL: **starvation freedom**.

We check that whenever a philosopher becomes hungry, he will eventually eat.

➡ LTL property: $\Box (hungry_i \Rightarrow \Diamond eat_i)$ for all philosophers i .

➡ In our model: $\Box ((hungry_0 \Rightarrow \Diamond eat_0)$ and
 $(hungry_1 \Rightarrow \Diamond eat_1)$ and
 $(hungry_2 \Rightarrow \Diamond eat_2)$ and
 $(hungry_3 \Rightarrow \Diamond eat_3)$ and
 $(hungry_4 \Rightarrow \Diamond eat_4))$

$hungry_i$: philosopher i is waiting to eat
 eat_i : philosopher i is eating

$\Box$ means always
 $\Diamond$ means eventually

This property may fail due to unfair executions, where some philosophers are **indefinitely postponed**.

To address this, we assume fair scheduling, meaning that every philosopher gets a chance to act.

We then enable **SPIN's built-in weak fairness**.

However, in this model a philosopher may be enabled to eat infinitely often, but **not continuously from some point on**. Therefore, weak fairness does not guarantee starvation freedom.



Dining philosophers in SPIN – Final solution

A possible solution to avoid deadlock and to ensure starvation freedom is the following:
each stick alternates which philosopher is allowed to use it.

- ➡ We define two possible states for each stick i :
- in state $available_i$, only philosopher i can take the stick.
 - in state $available_{i+1}$, only philosopher $i+1$ can take the stick

At any time, each stick is in exactly one of these states.

- ➡ After a philosopher releases a stick, the stick switches its state, giving access to the other adjacent philosopher.

This alternation removes uncontrolled competition on resources, even though philosophers may choose the first stick **nondeterministically** (as in the initial model).

In our example, $stick_0$, $stick_2$ and $stick_4$ start in $available_i$, while $stick_1$ and $stick_3$ start in $available_{i+1}$.



Dining philosophers in SPIN – Repository

The full PROMELA models used in this demo are available in the *demo* folder of the project repository

➡ github.com/matti8702/formal_methods_final_presentation_SPIN

Includes:

- **deadlock** verification model

➡ *demo/1_deadlock_detection/philosophers_with_deadlock.pml*

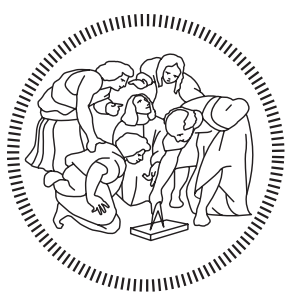
- **starvation-free** verification model

➡ *demo/2_starvation_free_verification/philosophers_deadlock_free.pml*

- **deadlock-free and starvation-free** model

➡ *demo/3_final_solution/philosophers_solution.pml*





POLITECNICO
MILANO 1863

Bibliography

➡ Slides

- Formal methods for software engineering (UIC 540) (San Pietro Pierluigi) [2025-26]
Section: [1.Model checking](#)

➡ Papers

- The Model Checker SPIN - Gerard J. Holzmann [1997]
Sections: [2.History](#) - [4.Introduction to SPIN](#) - [5.Büchi automaton in SPIN](#)
- End to End Verification and Validation with SPIN - Asankhaya Sharma [2013]
Section: [3.Real world applications](#)
- Mars Code - Gerard J. Holzmann [2014]
Section: [3.Real world applications](#)
- PACEMAKER System Specification – Boston Scientific [2007]
Section: [3.Real world applications](#)
- On Nested Depth First Search – Gerard J. Holzmann [1997]
Section: [6.Search algorithms](#)

➡ Papers

- Automating software feature verification - G.J. Holzmann, Margaret H. Smith [2000]
Section: [3.Real world applications](#)

➡ Official resources

- [Github Repository of SPIN](#)
- [SPIN Official Website](#)

➡ Books

- The SPIN Model Checker - Primer and Reference Manual - G.J. Holzmann [2003]
Sections: [2.History](#) - [4.Introduction to SPIN](#) - [6.Search algorithms](#) - [7.PROMELA language](#)

➡ Other resources

- [ACM Software System Award - Gerard J. Holzmann](#)
Section: [2.History](#)